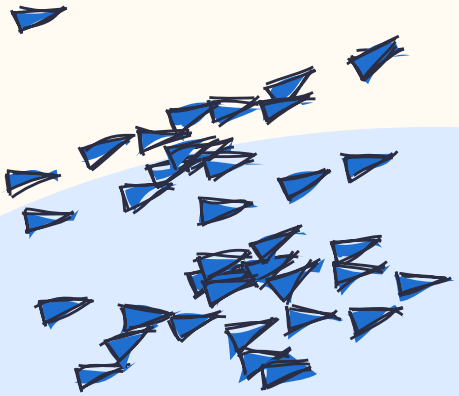
Tech Leaders · Class 20

Processing: Flocking Simulation (Boids)

No leader. Three rules. Beautiful chaos.



Emergence | Vectors | 3 rules | Experiments

Did you know?
Craig Reynolds invented 'boids' in 1986. His idea later won an Academy Award for technical achievement and is used in films for bats, birds and crowds.

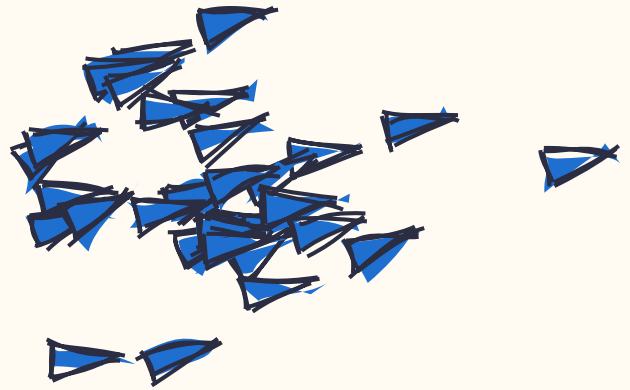
LEARN

Let's think...

Who is leading the flock?

Thousands of starlings

At sunset, thousands of starlings swirl as one giant shape, turning in a split second. Fish schools split around sharks and rejoin. Is one bird giving orders? Or is something smarter going on?



The front bird!

They're telepathic?

Simple rules!

Today: you code **boids** in Processing: each bird follows 3 local rules, and a flock **emerges**.

Learning objectives

By the end of this class I can...

1	**Emergence:** explain how **simple local rules** create complex group behaviour with no leader	**Proof:** I can explain a murmuration without saying 'the leader bird'
2	**Vectors:** use **PVector** for position, velocity, acceleration and **steering force**	**Proof:** my single boid smoothly steers towards the mouse
3	**Three rules:** code **separation, alignment and cohesion** with a neighbour radius and weights	**Proof:** my 150 boids flock on screen
4	**Experiment:** change **weights and radius** scientifically and measure the effect (and the n^2 cost)	**Proof:** my table links each parameter change to a behaviour

LEARN

Objective 3 · Three rules · video

Video time!



How do Boids Work? A Flocking Simulation

dante · about 4 minutes · youtu.be/QbUPfMXXQIY

Watch for...

1. What are the **three rules** each boid follows?
2. Does any boid know where the **whole flock** is going?
3. What happens when one rule gets stronger?

Scan to watch

After the video: Which of the three rules do you think matters most? Predict, then we'll test it.

LEARN

Objective 1 · Emergence

Emergence: simple rules, complex result

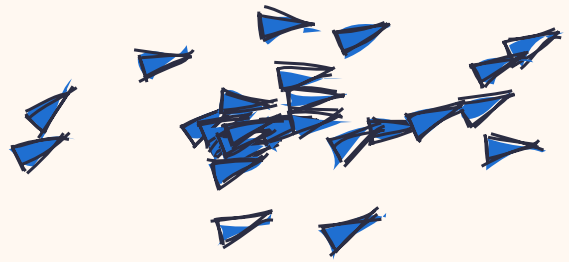
The idea

Emergence: complex group patterns arise from simple **local** rules, with **no leader** and no master plan.

Each boid only sees neighbours within a small **radius** (say 50 px). It never sees the whole flock. Yet swirling, splitting and merging appear.

Same idea: traffic jams, ant trails, crowds leaving a stadium, even the internet.

Example: a stadium wave



No one controls a Mexican wave at a cricket match. Each person just stands when their neighbour does. The wave 'emerges' and travels round the whole stadium.

LEARN

Objective 2 · Vectors

Vectors: position, velocity, acceleration

The idea

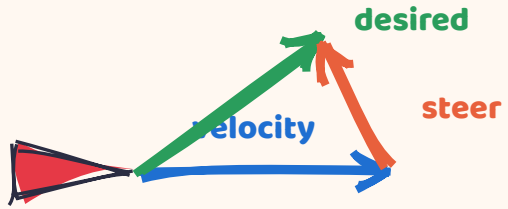
A **PVector** has x and y: a position, a velocity (direction + speed), or a force.

Each frame: **velocity += acceleration**; **position += velocity**; acceleration = 0.

Steering (Reynolds): **steer = desired − velocity**, then **limit** it to maxForce. Boids turn **gradually**, like real animals, instead of snapping.

Useful: add(), sub(), mult(), setMag(), limit(), dist(), heading().

Example: a boat turning



A boat can't instantly face a new direction. The rudder pushes a little each moment and the boat curves round. maxForce is the rudder's strength.

Objective 3 · Three rules

The three rules

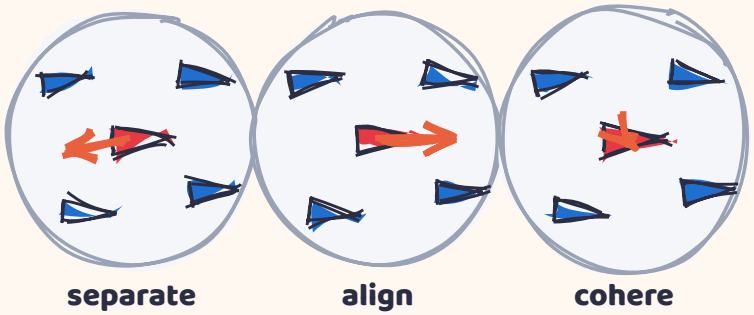
The idea

For each neighbour within the radius:

1. **Separation**: steer **away** from neighbours that are too close (avoid crowding).
2. **Alignment**: steer towards the **average heading** of neighbours.
3. **Cohesion**: steer towards the **average position** (centre) of neighbours.

Add the three forces with **weights**: e.g. sep × 1.5, ali × 1.0, coh × 1.0.

Example: walking with friends



Walking home in a group: don't bump into anyone (separation), walk the same way (alignment), don't drift off alone (cohesion). You already follow boid rules!

LEARN

Objective 4 · Experiment

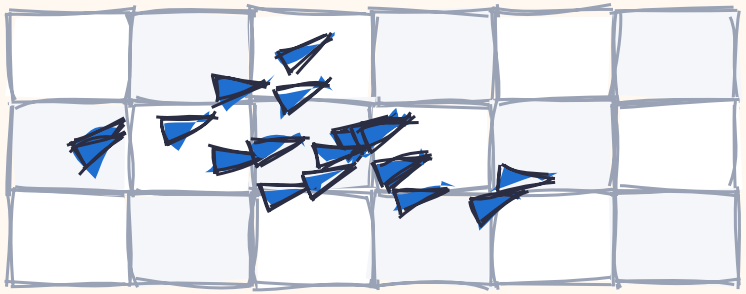
Experiment like a scientist (and mind the cost)

The idea

Change **one parameter at a time** and record the behaviour: separation weight ↑ → spread-out flock; cohesion ↑ → tight balls; radius ↓ → many small flocks.

Cost: every boid checks every other boid → **n^2** comparisons. 100 boids = 10 000 checks per frame; 1000 boids = 1 000 000! Real simulations use **spatial grids** so each boid checks only nearby cells.

Example: a class register vs a seating plan



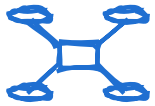
To find who's near you, checking all 1000 students is slow. Looking only at your row (a grid cell) is fast. Same answer, far less work.

Real world

Where flocking rules fly



Film effects
Bat swarms and crowds in movies use boid-style rules



Drone light shows
Hundreds of drones fly in formation without collisions



Ecology
Scientists model fish schools and bird migration



Crowd safety
Planners simulate crowds to design safer exits

? How could a crowd simulation make Hajj, a cricket final, or a busy railway station safer?

SDG 15: life on land
Simulations help scientists understand how habitat changes affect flocks and herds.

Quick quiz

Boid logic

Question	Answer
1. Name the 3 rules.	
2. steer = ? – ?	
3. Why limit the steering force?	
4. Separation weight very high. What happens?	
5. 200 boids, naive checks per frame?	

Answers

Cover! 1: separation, alignment, cohesion · 2: desired – velocity · 3: smooth, realistic turning · 4: boids spread out, no tight flock · 5: 40 000

Part 2 · Make it

Now let's put it to work!

You know the ideas. Time to play, build and test. Use these slides while you work:

1. Human boids
2. Our simulation lab
3. The Boid class: vectors + steering
4. The three rules in code
5. Run experiments
6. Experiment table
7. Simulation scientist challenges

Class game

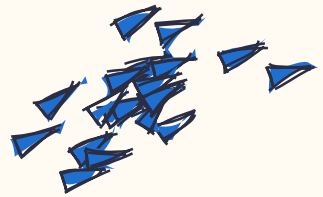
Human boids

How to play

1. 20 students in the hall. Each secretly picks a 'neighbour' person A and person B.
2. Rule 1: stay about 1 big step from everyone (separation).
3. Rule 2: walk the same direction as people around you (alignment). Rule 3: don't get left at the edge (cohesion).
4. Teacher calls 'shark!' and points: everyone near the point steps away. Watch the flock split and rejoin.

Then discuss

Did anyone lead? How did the 'shark' change the shape?



Big idea

Order without a leader: that's emergence.

MAKE

Build it

Our simulation lab

Processing 4
Java mode

PVector reference
processing.org/reference/PVector

Experiment sheet
One change per row

Starter code
Boid class in the class folder

Screen recorder
To compare behaviours

A partner
Predict, then test

Use a Boid class with an ArrayList: each boid owns its position, velocity and acceleration.

Save versions
Save as flock_v1, flock_v2... before big changes, so you can always go back.

MAKE

Objective 2 · Vectors · Build step 1 of 3

The Boid class: vectors + steering

```
ArrayList<Boid> flock = new ArrayList<Boid>();
float R = 50, maxSpeed = 3, maxForce = 0.05;
float wSep = 1.5, wAli = 1.0, wCoh = 1.0;

void setup() {
  size(900, 600);
  for (int i = 0; i < 150; i++) flock.add(new Boid(random(width), random(height)));
}

void draw() {
  background(15, 25, 45);
  for (Boid b : flock) { b.flock(flock); b.update(); b.edges(); b.show(); }
}

class Boid {
  PVector pos, vel, acc = new PVector();
  Boid(float x, float y) { pos = new PVector(x, y); vel = PVector.random2D().mult(2); }

  PVector steerTo(PVector desired) {                  // Reynolds steering
    desired.setMag(maxSpeed);
    return PVector.sub(desired, vel).limit(maxForce);
  }
  void update() { vel.add(acc).limit(maxSpeed); pos.add(vel); acc.mult(0); }
  void edges()  { pos.x = (pos.x + width) % width; pos.y = (pos.y + height) % height; }
  void show() {
    pushMatrix(); translate(pos.x, pos.y); rotate(vel.heading());
    fill(120, 200, 255); noStroke(); triangle(8, 0, -6, 4, -6, -4); popMatrix();
  }
```

Physics

update(): velocity += acceleration; position += velocity; reset acceleration.

Wrap

edges() makes boids leave one side and reappear on the other.

Next

flock() (next slide) adds the three rule forces to acc.

MAKE

Objective 3 · Three rules · Build step 2 of 3

The three rules in code

```
  void flock(ArrayList<Boid> all) {
    PVector sep = new PVector(), ali = new PVector(), coh = new PVector();
    int n = 0;
    for (Boid o : all) {
      float d = PVector.dist(pos, o.pos);
      if (o != this && d < R) {
        PVector away = PVector.sub(pos, o.pos).div(d * d);   // closer = stronger
        sep.add(away);
        ali.add(o.vel);
        coh.add(o.pos);
        n++;
      }
    }
    if (n > 0) {
      sep = steerTo(sep);
      ali = steerTo(ali.div(n));                          // average heading
      coh = steerTo(coh.div(n).sub(pos));                 // towards the centre
      acc.add(sep.mult(wSep)).add(ali.mult(wAli)).add(coh.mult(wCoh));
    }
  }
}   // end of class Boid
```

$1 / d^2$

Very close neighbours push away much harder than distant ones.

Averages

Alignment = average velocity; cohesion = average position minus mine.

Weights

wSep, wAli, wCoh: the dials for your experiments.

MAKE

Objective 4 · Build step 3 of 3

Run experiments

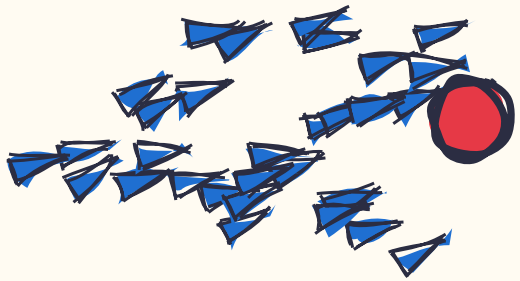
a Add keys: **1/2/3** raise wSep/wAli/wCoh, **shift**+key lowers; show values with text().

b Experiment table: change ONE value, predict, observe, record (tight / loose / lines / clumps).

c Measure **frameRate** at 150, 500 and 1000 boids. Plot FPS vs n.

d Add a **predator** (the mouse): boids within 100 px get a strong flee force.

e Add **obstacles**: circles the boids must steer around.



Check: Your table shows 5+ experiments and an FPS vs boids result.

Test like an engineer

Experiment table

Change	Prediction	What happened
wSep 1.5 → 3.0		
wCoh 1.0 → 2.5		
R 50 → 20		
FPS at 150 / 500 / 1000 boids		
Predator added: effect		

Debug checklist

Boids vanish? NaN: guard against d = 0 (o != this, and d > 0).
All spin in circles? maxForce too large.
Very slow? Too many boids for n^2: try a grid, or fewer boids.

A good experiment changes one thing and keeps everything else the same.

Level up!

Simulation scientist challenges

Choose your challenge



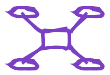
Spatial grid: bin boids into cells; only check neighbouring cells. Measure the speed-up.



Two species: fish that flock and sharks that hunt. Who survives?



Field of view: boids only see neighbours in front of them (dot product of heading).



3D: move to P3D and flock in three dimensions.

Think like an engineer

Real starlings track about 7 nearest neighbours, not everyone within a distance. How would that change your code?

Boss level

Make the simulation interactive art: boids leave trails that fade, and sound plays when they crowd.

Objective check

Can I really do it? Prove it.

01	Explain emergence using the flock: who is in charge?	○ ○ ○
02	Show steerTo(). Why desired – velocity, and why limit it?	○ ○ ○
03	Point to separation, alignment and cohesion in your code.	○ ○ ○
04	Show your experiment table and FPS vs boids. What does n^2 mean for performance?	○ ○ ○

Rate yourself

Colour one circle each. Red: not yet. Amber: with help. Green: I can explain it to a friend.

Teacher: Flock gallery: each pair shows its most surprising behaviour and the parameter values that caused it.

Show what you know

What did we learn?

- [x] Emergence: simple local rules, complex global patterns
- [x] PVectors: velocity += acceleration; position += velocity
- [x] steer = desired − velocity, limited by maxForce
- [x] Separation, alignment, cohesion with weights
- [x] Naive neighbour checks cost n^2; grids fix it

Exit ticket: which rule stops the boids from crashing into each other?

At home

Watch birds, ants or traffic for 5 minutes. Which boid rule do you see?



Next class:
Pathfinding Visualiser: BFS and A*!